Đệ quy và ngăn xếp lời gọi

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 3: Phân tích thuật toán và đệ quy.
- Các bài trong chương: Big-O, bất biến và kiểm tra thuật toán; Đệ quy và ngăn xếp lời gọi.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

Mục tiêu bài học

Xác định trường hợp cơ sở, bước tiến triển và chi phí của lời giải đệ quy.

Kiến thức nền

Hàm đệ quy gọi lại chính nó để giải bài toán nhỏ hơn. Cần có trường hợp cơ sở và đại lượng giảm tới trường hợp đó. Với giai thừa n không âm: 0! = 1 và n! = n*(n-1)!. Nếu không kiểm tra miền đầu vào, lời gọi với số âm có thể không bao giờ tới cơ sở.

Mỗi lời gọi lưu trạng thái trên ngăn xếp lời gọi. Tính giai thừa theo cách trên có n bước và độ sâu O(n). Đệ quy không tự làm thuật toán nhanh hơn; phiên bản lặp có thể giảm bộ nhớ phụ.

Một bài toán chia thành nhiều nhánh có thể lặp lại tính toán. Fibonacci đệ quy trực tiếp tính nhiều lần cùng giá trị; memoization lưu lời giải bài toán con để tái sử dụng. Phải phân biệt số lời gọi với độ sâu ngăn xếp. Trong thực tế, giới hạn ngăn xếp có thể làm đệ quy sâu thất bại.

1. Ba câu hỏi bắt buộc

Đầu vào nào hợp lệ? Khi nào giải trực tiếp? Mỗi lời gọi tiến gần cơ sở bằng cách nào? Thiếu một câu trả lời có thể gây không kết thúc hoặc xử lý sai biên. Không đặt cơ sở theo cảm tính chỉ để chương trình dừng.

2. Giai thừa và điều kiện miền

```
def factorial(n):
    if not isinstance(n, int) or isinstance(n, bool) or n < 0:
        raise ValueError('n must be a non-negative integer')
    def solve(k):
        if k == 0:
            return 1
        return k * solve(k - 1)
    return solve(n)
```

```
assert factorial(0) == 1
assert factorial(5) == 120
```

Ví dụ dùng số nhỏ để học; Python giới hạn độ sâu đệ quy. Với dữ liệu lớn nên lựa chọn cách lặp hoặc hàm thư viện phù hợp, không tự tăng giới hạn ngăn xếp để che thiết kế.

3. Truy vết thời gian và bộ nhớ

factorial(3) tạo các lời gọi 3,2,1,0 rồi trả ngược. Số phép nhân là ba, ngăn xếp sâu tuyến tính. Với Fibonacci trực tiếp, cây lời gọi có các nhánh lặp; độ sâu không bằng tổng số nút trong cây. Memoization giảm tính lại bằng bộ nhớ lưu kết quả.

4. Bài tập có hướng giải

Viết tổng danh sách bằng đệ quy với chỉ số thay vì cắt danh sách ở mỗi bước. Cơ sở index==len(values) trả 0; bước tiếp cộng values[index]. Phân tích O(n) thời gian và O(n) ngăn xếp trong mô hình phép cộng hằng số. Kiểm thử danh sách rỗng, âm và một phần tử; ghi rõ giả định về kích thước số nguyên nếu phân tích chi phí bit.

> **Bài tập vận dụng bổ sung**
>
> Viết giả mã giai thừa có kiểm tra n>=0. Vẽ cây lời gọi Fibonacci(5) và đánh dấu các giá trị được tính lại; đề xuất cách lưu kết quả.

Hướng giải bài tập bổ sung

Có cơ sở, miền đầu vào và bước tiến triển rõ ràng. Memoization tránh tính lại, còn vòng lặp có thể tránh ngăn xếp sâu cho giai thừa.

Câu hỏi tự kiểm tra

1. Đệ quy kết thúc thường cần gì?
2. Giá trị 0! theo định nghĩa thông thường là gì?
3. Ngăn xếp lời gọi lưu gì?
4. Memoization giúp gì?
5. Đệ quy có luôn nhanh hơn lặp không?

Gợi ý đáp án và điểm cần nhớ

- Cơ sở và bước tiến tới cơ sở. Bài toán phải giảm tới tình huống giải trực tiếp.
- 1. 0! = 1 làm cơ sở cho công thức giai thừa.
- Trạng thái các lời gọi chưa hoàn tất. Lời gọi con trả về để tiếp tục lời gọi cha.
- Tái sử dụng kết quả bài toán con. Các trạng thái đã giải được lưu lại.

- Không. Thời gian và bộ nhớ phụ thuộc cách phân rã và triển khai.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html